

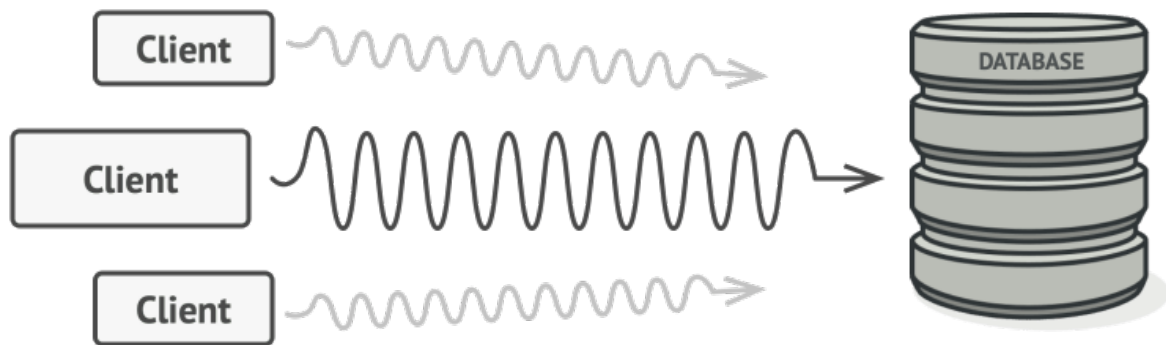
1 Nazwa

Pełnomocnik

2 Problem

Wzorzec stosuje się w sytuacjach gdy bezpośredni dostęp do obiektu jest kosztowny ograniczony lub wymaga dodatkowej kontroli Przykład:

- obiekt zajmuje dużo pamięci i powinien być tworzony tylko wtedy gdy jest potrzebny
- Dostęp do obiektu wymaga sprawdzenia uprawnień
- obiekt znajduje się na zdalnym serwerze

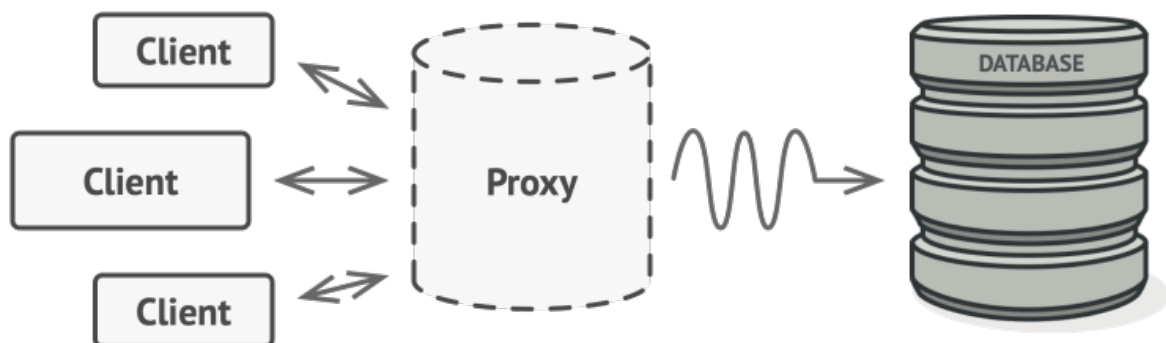


Rysunek 1: Bazy danych potrafią być powolne

3 Rozwiązanie

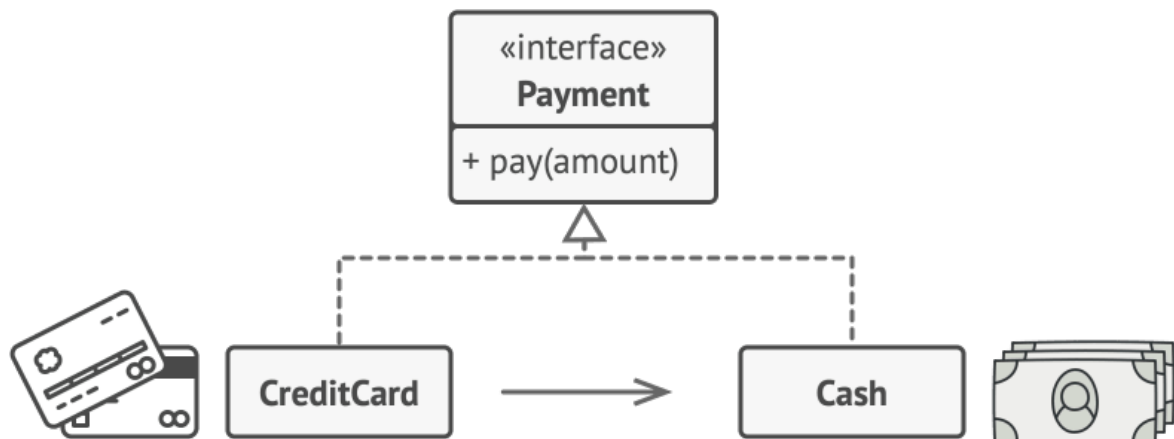
Wprowadzenie obiektu pośredniczącego który implementuje ten sam interfejs co oryginalny.

- Interfejs - wspólny dla pełnomocnika i obiektu rzeczywistego
- Pełnomocnik - przechowuje referencje do obiektu rzeczywistego kontroluje dostęp do niego i może zarządzać jego cyklem życia.



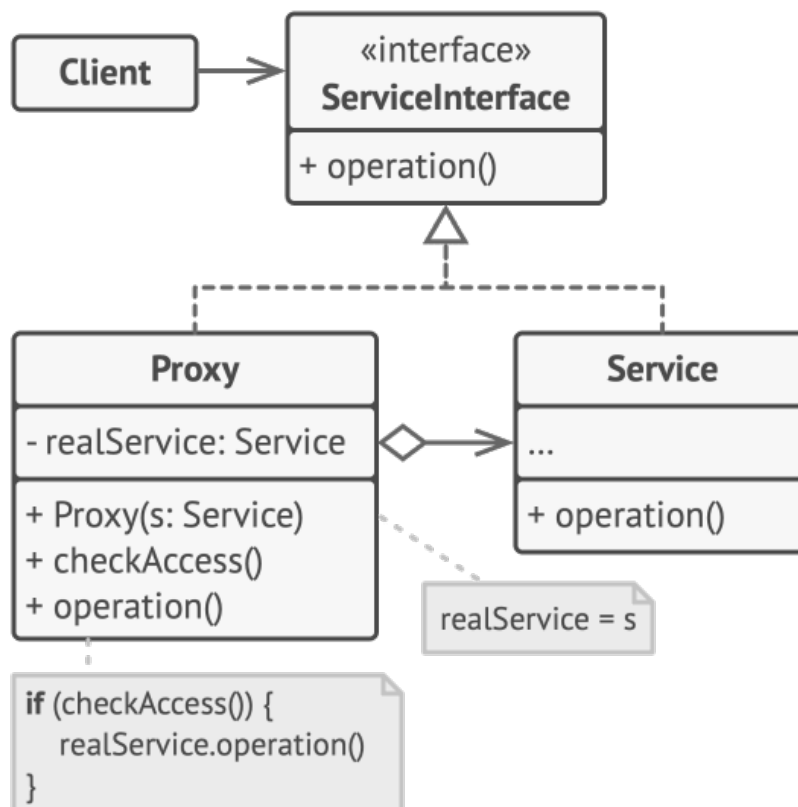
Rysunek 2: Pełnomocnik udaje obiekt bazy danych. Może być odpowiedzialny za inicjalizację czy "cacheowanie" bez wiedzy klientów czy nawet bazy danych

3.1 Przykład z życia



Karta płatnicza jest pełnomocnikiem dla konta bankowego, a ona sama jest pełnomocnikiem gotówki. Oba mają ten sam interfejs: mogą zostać użyte do płacenia. "klient" czuje się dobrze, bo nie musi nosić ze sobą portfela wypełnionym pieniędzmi, a sprzedawca nie musi się bać, że zostanie okradziony w drodze do banku.

3.2 Struktura



1. **ServiceInterface** określa interfejs naszego serwisu. pełnomocnik musi z niego korzystać by móc podszyc się pod **Service**
2. **Service** to klasa, którą nasz pełnomocnik udaje.

3. Klasa **Pelnomocnika** posiada referencje do naszego realnego obiektu serwisu po tym jak skonczy procesowanie wszystkiego(np. kontrola dostepu) przekazuje zapytanie dalej do realnego obiektu. Pelnomocnik zarzadza calym cyklem zycia obiektu.

4 Konsekwencje

4.1 Zalety

- Pozwala na optymalizacja
- Zwiększa bezpieczenstwo przez kontrole dostepu

4.2 Wady

- moze zwiekszyc poziom skomplikowana kodu
- Wprowadza dodatkowa warstwe co moze opoznic czas odpowiedzi